Trajectory Optimisation

Contents

1. Overview 2
2. From a path to a trajectory 4
   - 2.1 What a trajectory is 4
   - 2.2 The best trajectory: cost and constraints 6
3. Turning functions into numbers: transcription 8
4. Single shooting: simulate, measure the miss, adjust 9
   - 4.1 Fire, see where it lands, adjust: one number by hand 9
   - 4.2 Shooting the drive to the door 12
   - 4.3 The long lever of the first control 14
5. Multiple shooting: shorter pieces, joined up 14
6. Direct collocation: the motion rules as equations at points 16
   - 6.1 States at points instead of a simulation 16
   - 6.2 The trapezoid rule by hand 16
   - 6.3 Checking the method on the straight drive 18
   - 6.4 The drive to the door 19
7. iLQR and DDP: better steps from the model 22
8. Direct and indirect methods 24
9. Summary 27
10. Sources 28
11. Key terms 29

Where this fits:

- **Builds on:** Gradient descent; Kinodynamic planning and phase space.
- **Compare with:** Motion primitives and lattice search.

1. Overview

Key point: A path says where the robot goes; a trajectory also says when, and with which commands. Trajectory optimisation writes the commands over time as a list of numbers and improves the list with gradients until the robot ends where it should, within its limits, at the least cost: the robot reaches the door pose in 6.18 s, against 8.08 s for stopping and turning along a route.

The robot can already find its way to the door. The planners of this chapter give it a route round the table, and planning with motion limits gives it drivable motions built from fixed pieces: arcs, straight runs and speed changes that respect how fast its wheels can turn and how fast it can speed up. Fixed pieces have a price: on a straight 3.0 m drive from rest to rest, a lattice search over such pieces, with steps of 0.5 s, drives it in 7.0 s, while the true least time is 6.2 s: speed up at the limit, cruise at the top speed, brake.

The drive the robot needs is not straight. The robot starts at rest at (2.0, 1.0) facing 30°, and must stop at the door pose, (3.6, 3.5) facing 90°, just below the door. Its limits are those of the same robot: a disk of radius 0.25 m, each wheel at most 0.6 m/s, its forward speed changing by at most 0.5 m/s every second. Three things stand in the way:

- **The table.** The straight line from start to door pose, 2.968 m long, passes 0.185 m from the table's corner; the disk needs 0.25 m, so it would hit the table by 0.065 m (notebook).
- **The turns.** The robot faces 30° at the start, the line runs at 57.4°, and the door pose faces 90°.
- **The timing.** A route says where to drive, not how fast at each moment. Following a route with one corner, at (3.0, 3.3), and stopping to turn at the corner takes 8.08 s (notebook).

Figure 1 plays the stop-and-turn drive and the drive this Note finds side by side, with a clock: watch the grey robot stop at the corner to turn while the blue one keeps moving round the table's corner.

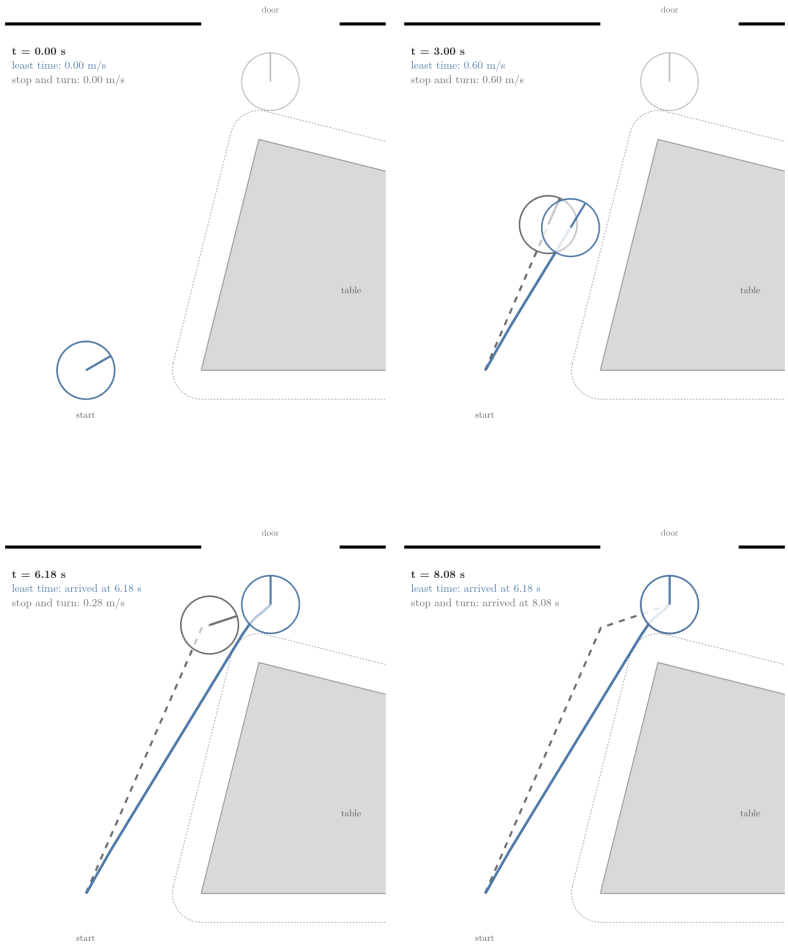


Figure 1: Two drives from rest at (2.0, 1.0) facing 30° to rest at the door pose (3.6, 3.5) facing 90°, past the table, one frame per 0.2 s. Grey: stop and turn, along a route with one corner at (3.0, 3.3): turn on the spot, drive, stop, turn, drive, stop, turn: 8.08 s. Blue: the least-time trajectory found in this Note: 6.18 s. Each robot is a disk of radius 0.25 m with a line for its heading; the dotted line round the table is the band the disk's centre must stay out of; the pale circle is the door pose. The last frame holds at 8.08 s

The blue drive arrives 1.90 s sooner. The blue robot never stops: it turns towards the door while it speeds up, runs at the 0.6 m/s limit, passes the table's corner with no room to spare, and turns to face the door as it brakes. No fixed set of pieces had to contain that motion; it was computed. This Note builds that computation:

- what a trajectory is, and the problem of finding the best one (Section 2);
- turning commands over time, which are functions, into a list of numbers (Section 3);
- single shooting: simulate the drive, measure the miss, adjust the commands (Section 4);
- multiple shooting: the same with the drive cut into pieces (Section 5);
- collocation: no simulation, the motion rules written as equations at points (Section 6);

- iLQR and DDP, which use the robot's model to take much better steps (Section 7);
- the other family, indirect methods, and Pontryagin's principle (Section 8).

2. From a path to a trajectory

> **Key point:** A trajectory gives the robot's state at every moment, and the commands that produce it. Finding the best trajectory means choosing the commands over time that minimise a cost, here the time or the effort, while the robot obeys its motion rules, starts and ends where it must, and keeps its limits.

The opening drive failed on timing as much as on the table: a route says where to drive, and the robot still needs to know how fast to go and how hard to turn at each moment.

2.1 What a trajectory is

The idea. A path is a line on the floor. A trajectory is the same line with a clock on it: where the robot is at 1 s, at 2 s, at 3 s, how fast it goes and how it is turning. Figure 2 shows the drive of Figure 1 this way: the dots, one every 0.5 s, are far apart where the robot is fast and close together where it speeds up or brakes.

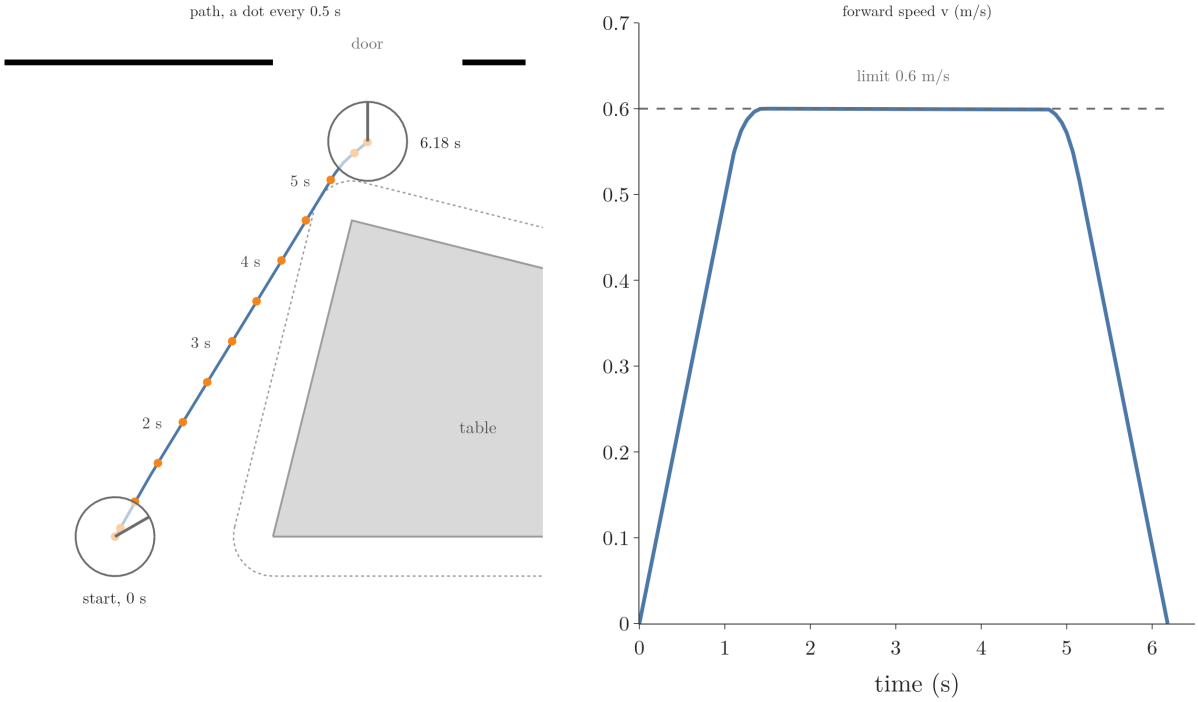


Figure 2: The least-time drive as a trajectory. Left: its path from the start (2.0, 1.0) to the door pose (3.6, 3.5), with a dot every 0.5 s and the time written at 2, 3, 4 and 5 s; the robot arrives at 6.18 s. Dots far apart mean the robot is fast. Right: its forward speed v over time: up at 0.5 m/s^2 for 1.2 s, along the 0.6 m/s limit (dashed), and down at 0.5 m/s^2 at the end

To describe the robot at one moment we need four numbers, its **state** (G-2314), everything about the robot that matters for what happens next, and two commands, its **control inputs** (G-2298). With their values at the start:

- x, y, the position of the robot's centre, in m (2.0 and 1.0);
- θ, its heading, the direction it faces, in rad (30°, 0.524 rad);
- v, its forward speed, in m/s (0 at the start);
- a, the acceleration, how fast v changes, in m/s^2 (a control input);
- ω, the turn rate, how fast θ changes, in rad/s (a control input).

The rules that say how the state changes are the robot's kinematic model (G-2297), plus one line that makes the speed change only through the acceleration. A dot over a symbol means how fast that quantity changes, per second:

$$\dot{x} = v \cos\theta$$

$$\dot{y} = v \sin\theta$$

$$\dot{\theta} = \omega$$

$$\dot{v} = a$$

The speed is part of the state, not a command, because the robot cannot change its speed at once: at most 0.5 m/s every second (the acceleration limit).

The wheel limit. Each wheel turns at most 0.6 m/s. With the wheel separation $L = 0.2$ m, the wheel speeds for a forward speed v and turn rate ω are, with:

- v_R, v_L, the right and left wheel speeds, in m/s;
- L, the wheel separation, 0.2 m.

$$v_R = v + \omega L/2$$

$$v_L = v - \omega L/2$$

Both must stay at most 0.6 m/s. Driving at 0.5 m/s:

$$0.5 + \omega \times 0.1 \leq 0.6$$

$$\omega \leq 1 \text{ rad/s}$$

and at rest the robot can spin at up to 6 rad/s. Turning costs speed: the faster the robot drives, the slower it can turn.

The standard term. A description of the robot's state at every moment of a time interval, together with the controls that produce it, is a **trajectory** (G-2886) (Kelly 2017; Lynch and Park §10.7).

2.2 The best trajectory: cost and constraints

Among all the trajectories that take the robot from the start to the door pose, we want the best one. "Best" needs a number, the cost. This Note uses two costs:

- **least time:** the duration T, in s; the opening drive asks for it;
- **least effort in a fixed time:** drive to the door pose in exactly 8 s, as gently as possible, with the effort defined below. A gentle drive is easier on the motors and the wheels' grip, and least effort in a fixed time is a standard first example: a cart that swings a pole upright with the motor force squared, added up over the swing, as small as possible (Kelly 2017).

With:

- T, the duration of the drive, in s (8 s for the effort problem);
- J, the effort, in $\mathrm{m}^2/\mathrm{s}^3$: the acceleration squared plus 0.01 times the turn rate squared, added up over the drive, so that turning at 1 rad/s counts like speeding up at 0.1 $\mathrm{m/s}^2$; squaring makes large commands cost much more than small ones, so the drive comes out smooth (Kelly 2017).

$$J = \int_0^T \left(a^2 + 0.01\,\omega^2\right) dt$$

The $\int$ sign adds up the quantity after it over every moment from 0 to T, the area under its curve (G-957).

The rules the trajectory must obey are its **constraints** (G-456), conditions every acceptable answer must meet:

1. the motion rules of Section 2.1, at every moment;
2. the start, (2.0, 1.0) facing 30° at rest, and the end, (3.6, 3.5) facing 90° at rest;
3. the limits at every moment: both wheels at most 0.6 m/s, the acceleration between -0.5 and 0.5 $\mathrm{m/s}^2$, the speed at least 0;
4. the table: the disk's centre at least 0.25 m from it.

Constraints of the first kind come from the robot's model, the second fix its ends, and the third and fourth hold along the whole drive. Finding the controls over time, and the duration if it is free, that minimise the cost while every constraint holds is **trajectory optimisation** (G-2887) (Kelly 2017; Lynch and Park §10.7). In symbols (Lynch and Park §10.7), with:

- $\mathbf{x}$, the state, the four numbers (x, y, θ, v), at the start (2.0, 1.0, 0.524, 0);
- $\mathbf{u}$, the controls, the two numbers (a, ω);

- f, the four motion rules of Section 2.1, which give how fast each state number changes.

$$\text{find } a(t), \omega(t), T$$

$$\text{minimising } T \text{ or } J$$

$$\text{with } \dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u})$$

$$\mathbf{x}(0) = \text{start}$$

$$\mathbf{x}(T) = \text{door pose}$$

$$\text{and the limits at every } t$$

Why one trajectory, not a rule for every state. The best possible answer would be a policy (G-1512): the best controls from every state the robot could be in. A policy needs a value for every state, and the states grow fast with their number: a walking robot with 10 joints has a state of 20 numbers, an angle and a turning speed for each joint, and the cost of covering every state grows exponentially with that number (Kelly 2017). Even our 4-number state, cut into 5 cm, 5° and 0.05 m/s steps, gives 140 positions across the room, 80 along it, 72 headings and 13 speeds:

$$140 \times 80 \times 72 \times 13$$

$$= 10483200 \text{ states}$$

for one room. Trajectory optimisation gives up on every state and finds one good trajectory, from this start to this goal, which needs far less computation (Kelly 2017). The price is that it answers only for this start: a robot pushed off the trajectory needs a new one, or a controller that brings it back.

The problem is stated. But $a(t)$ and $\omega(t)$ are functions, with a value at every one of infinitely many moments, and a computer can only search over a list of numbers.

3. Turning functions into numbers: transcription

> **Key point:** Cut the drive into short intervals and keep a few numbers per interval: the controls, or the controls and the states. The trajectory problem becomes a problem of choosing a list of numbers: the 8 s drive with 30 intervals has 60 control numbers.

Section 2 left the controls as functions of time, with infinitely many values to choose.

The idea. A film is a list of still pictures, 24 every second, and the eye fills in between. A drive can be described the same way: cut the 8 s into 30 intervals of 0.267 s and hold the acceleration and the turn rate constant in each. The two functions become 30 pairs of numbers, 60 in all (Figure 3).

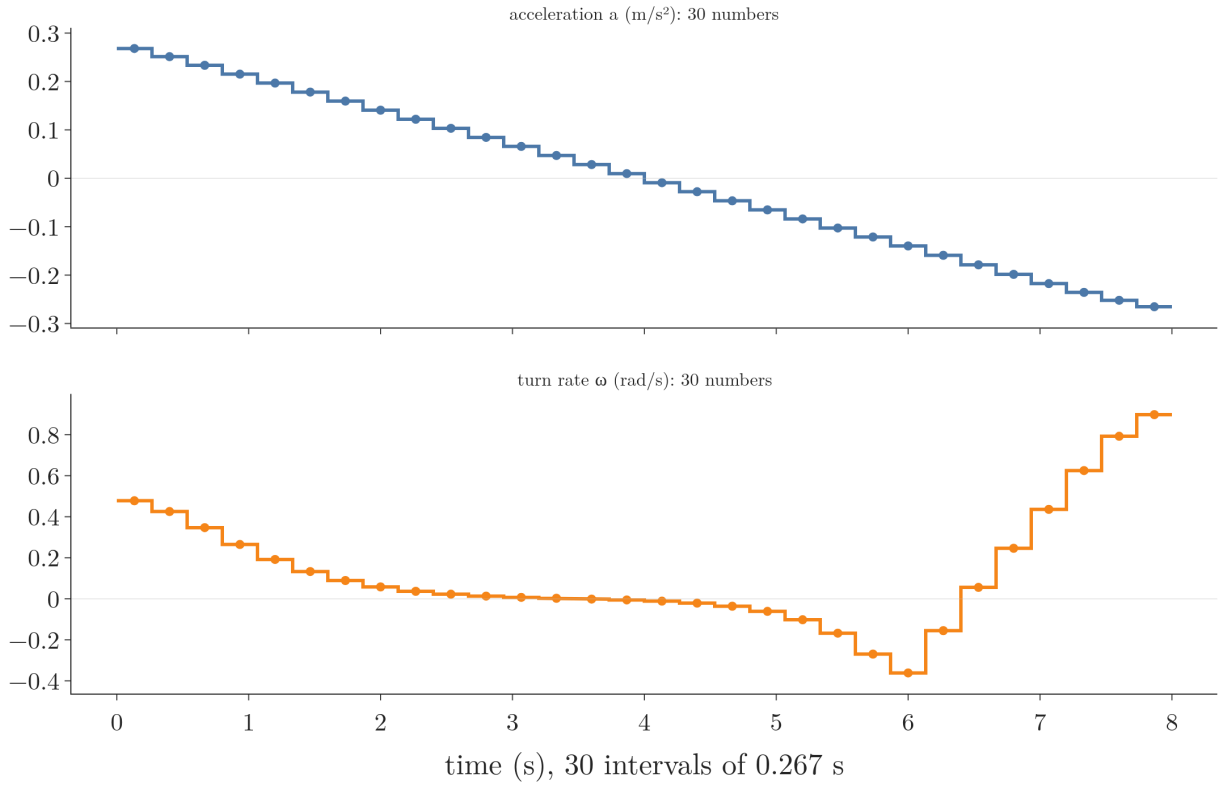


Figure 3: The controls of the 8 s least-effort drive (found in Section 4) as a list of numbers. The 8 s are cut into 30 intervals of 0.267 s, and in each the acceleration a (top) and the turn rate ω (bottom) are held constant: $30 + 30 = 60$ numbers. The acceleration falls in nearly a straight line from 0.27 to -0.27 m/s^2: speed up, then brake

The standard terms. Turning a trajectory optimisation problem, whose unknowns are functions, into a problem whose unknowns are a finite list of numbers is **transcription** (G-2888) (Kelly 2017). The result is an optimisation over a list of numbers: choose the numbers to minimise a cost, subject to equations and inequalities that may be curved, such as the cosine in $\dot{x} = v\cos\theta$ (Kelly 2017).

There are three ways to pick the numbers (Lynch and Park §10.7):

1. **the path only**, and work out the controls from it; this does not work for our robot, which has 2 controls for its 3 position numbers x, y, θ (Lynch and Park §10.7), so most paths cannot be driven;
2. **the controls only**, and find the states by simulating the motion rules: shooting (Section 4);
3. **the controls and the states**, and make the motion rules hold between them as equations: collocation (Section 6).

Discretising first and optimising after is what makes all of these methods direct; the other family, indirect methods, comes in Section 8 (Kelly 2017). The list of numbers is ready; the next question is how to improve it.

4. Single shooting: simulate, measure the miss, adjust

> **Key point:** Guess the controls, simulate the drive, measure how far it ends from the goal, and change the controls against the gradient of that miss; repeat. On the straight drive each step cuts the miss to a quarter, and on the room drive 60 numbers reach the door pose in 73 iterations, but an early control moves the end far more than a late one, which makes long drives hard.

Section 3 turned the controls into numbers. Shooting keeps only those numbers as unknowns and finds the states by simulation.

4.1 Fire, see where it lands, adjust: one number by hand

The idea. To hit a target with a cannon, fire, see where the ball lands, change the launch speed and fire again (Kelly, cannon). A drive works the same way: choose the controls, simulate the drive, and see where it ends.

We start with the straight drive, which has one number to choose. The robot must cover 3.0 m along a line in 10 s, from rest to rest: speed up at a for 5 s, then brake at a for 5 s. With:

- a, the acceleration, in m/s^2, the one number we choose (first guess 0);
- s_{end}, the distance covered after 10 s, in m;
- e, the miss, s_{end} minus the 3.0 m wanted, in m.

After 5 s the speed is $5a$. The distance in each half follows from constant acceleration:

$$\text{first half} = a \times 5^2/2$$

$$= 12.5\,a$$

$$\text{second half} = 5a \times 5 - a \times 5^2/2$$

$$= 12.5\,a$$

$$s_{\text{end}} = 25\,a$$

The braking takes the speed from $5a$ back to 0, so every shot stops; only the distance can miss.

The miss and its slope. The first shot, $a = 0$, does not move: it misses by 3.0 m. To improve it we use gradient descent (G-862): take the squared miss as the cost and step against its slope. With:

- e, the miss, $25a - 3$, in m;
- C, the cost, the squared miss, e^2;
- η, the learning rate (G-1068), the size of each step, 0.0006.

The slope of the squared miss is twice the miss, and the miss grows by 25 m for every 1 m/s^2 of a, so by the chain rule (G-371) the slope of C with respect to a is

$$\frac{dC}{da} = 2e \times 25$$

$$= 50\,e$$

The first step, from $a = 0$ with $e = -3$:

$$\frac{dC}{da} = 50 \times (-3)$$

$$= -150$$

$$a = 0 - 0.0006 \times (-150)$$

$$= 0.09$$

The second shot ends short again:

$$s_{\text{end}} = 25 \times 0.09$$

$$= 2.25 \text{ m}$$

$$e = 2.25 - 3$$

$$= -0.75 \text{ m}$$

Figure 4 fires the shots one per frame; watch the red miss shrink.

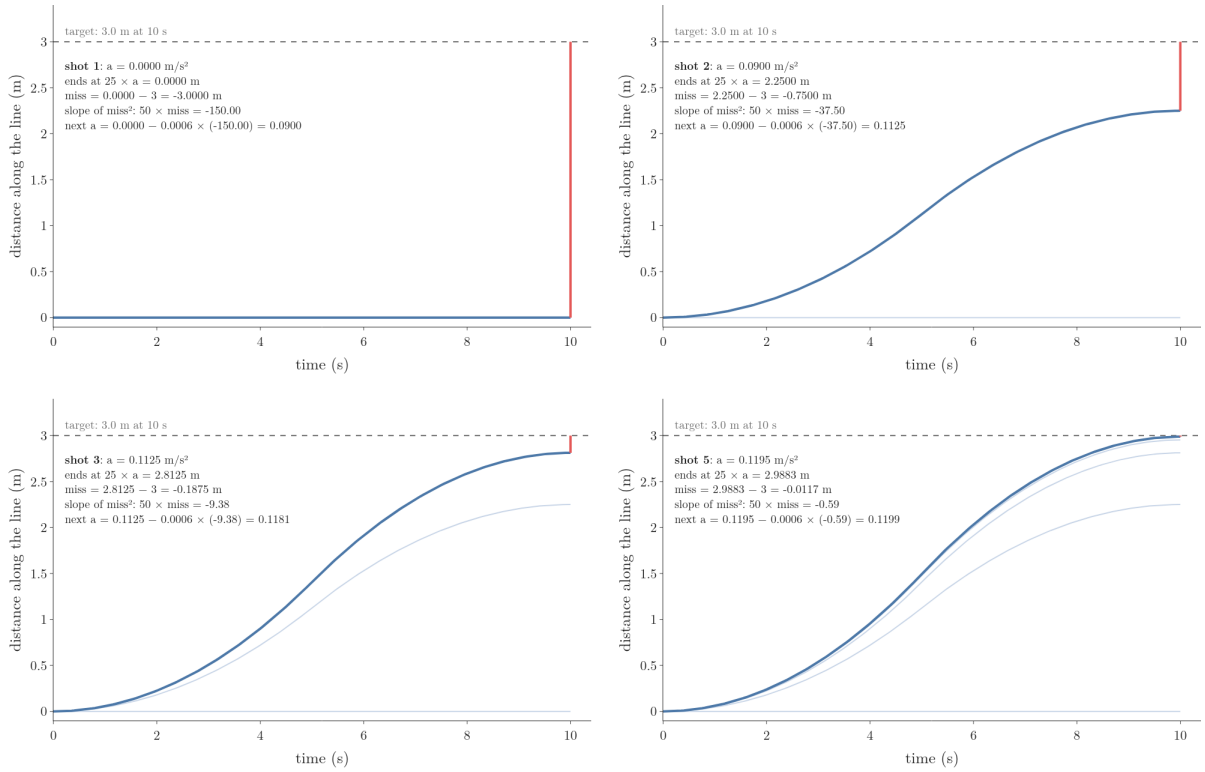


Figure 4: Single shooting with one number, the acceleration a, on the straight drive: speed up at a for 5 s, brake at a for 5 s, ending 25a metres along. Each frame fires one shot (blue: distance along the line over time; earlier shots pale), measures the miss at 10 s against the target 3.0 m (dashed; the miss in red), and steps a against the slope of the squared miss, 50 times the miss, with step size 0.0006. a goes 0, 0.09, 0.1125, 0.1181, 0.1195; the miss 3.0, 0.75, 0.1875, 0.0469, 0.0117 m

Each step cuts the miss to a quarter: 3.0, 0.75, 0.1875, 0.0469 m (notebook). The quarter follows from one step, with e_{new} the miss after it:

$$e_{\text{new}} = 25\,(a - 0.0006 \times 50\,e) - 3$$

$$= 25\,(a - 0.03\,e) - 3$$

$$= (25a - 3) - 0.75\,e$$

$$= e - 0.75\,e$$

$$= 0.25\,e$$

The shots close in on the a that makes the miss 0:

$$a = 3/25$$

$$= 0.12 \text{ m/s}^2$$

which reaches 0.6 m/s at 5 s, exactly the robot's top speed.

The standard terms. Keeping only the controls (and the start) as unknowns, finding the trajectory by one simulation from the start, and adjusting the controls until the end meets the goal is **single shooting** (G-2890), also called direct shooting (Kelly 2017; Kelly, cannon). The gap between where a simulation ends and where it must end is a **defect** (G-2891) (Kelly, cannon); the solver drives every defect to 0.

4.2 Shooting the drive to the door

The room drive has 60 numbers instead of one, and turning makes the motion rules curved. The loop is the same:

1. guess the 60 controls; our first shot faces the goal and speeds up and brakes too gently, ending 1.048 m short (notebook);
2. simulate the drive from the start, in steps of 0.067 s;
3. measure the miss at the end and how far the drive breaks a limit or enters the table's band;
4. change all 60 controls at once, using how each miss and each broken limit changes with each control;
5. repeat until nothing misses and nothing is broken.

Each simulation step uses a fourth-order Runge–Kutta step, a more accurate version of the Euler step (G-2299) that samples the motion rules four times per step (Kelly 2017).

The table that lists how every miss changes with every control is the Jacobian (G-980). Plain gradient descent would step down the slope of the cost; we use SciPy's least-squares solver, a Gauss–Newton method, which uses the Jacobian to aim each step at the point where the misses, treated as straight lines, would all be 0, and so needs far fewer steps (Virtanen et al. 2020). The misses and broken limits are added to the cost, squared and weighted, and the weight is raised from 100 to 1,000,000 so that by the end they must be 0: such penalty terms

are the usual way to add constraints to shooting (Tedrake, Underactuated ch. 10). Figure 5 runs it; watch the end of the blue drive close on the door pose.

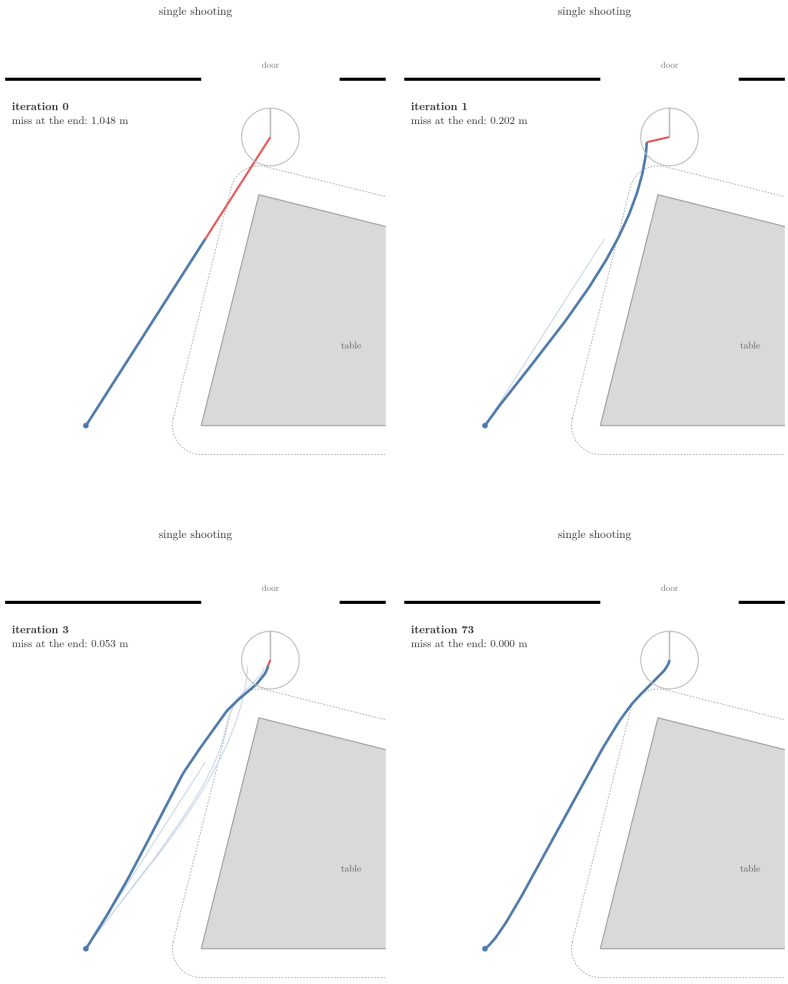


Figure 5: Single shooting of the 8 s least-effort drive to the door pose, one solver iteration per frame (thinning out after the first 12). Each frame is one simulation from the start with the current 60 controls (blue; the 4 before it pale), and the red line is the miss at the end. The first shot, facing the goal and too gentle, ends 1.048 m short; early iterations cut through the table's band while the penalty weight is low; after 73 iterations the drive ends on the door pose and passes the table's corner

After 73 iterations the drive ends on the door pose, missing it by less than 0.001 mm, and keeps every limit, touching the table's band to within 1 mm, with an effort of 0.217 (notebook). Its acceleration is the nearly straight falling line of Figure 3.

4.3 The long lever of the first control

Single shooting worked here, but it has a weakness that grows with the length of the drive. A control early in the drive acts on everything after it; a late control acts on little. On the straight drive with two numbers, a_1 for the first 5 s and a_2 for the last 5 s, the first half covers $12.5\,a_1$ as in Section 4.1 and ends at the speed $5a_1$, which the second half keeps adding for 5 s:

$$s_{\text{end}} = 12.5\,a_1 + 5a_1 \times 5 + 12.5\,a_2$$

$$= 37.5\,a_1 + 12.5\,a_2$$

The first number moves the end 3 times as far as the second. On the room drive, 0.1 rad/s more turn rate in the first interval moves the end 0.079 m, and the same change in the last interval moves it about 0.0001 m (notebook): a turn early on swings the whole rest of the drive round.

So the solver must balance numbers whose effects differ by orders of magnitude, and on long or sensitive problems the steps go wrong; this is a property of the problem, not of the program (Tedrake, Underactuated ch. 10). Limits along the way are also awkward, because the states are not unknowns: every limit is a long chain of simulation back to the controls (Kelly 2017). Single shooting suits problems with simple controls and few path constraints, such as spacecraft, and can fail on harder ones (Kelly 2017; Kelly, cannon). The fix is to shorten the lever.

5. Multiple shooting: shorter pieces, joined up

> **Key point:** Cut the drive into pieces, simulate each from its own start state, and make those start states unknowns too, with a defect at every join that must close. Each control then acts on one short piece: on the room drive with 5 pieces, the first turn rate's lever falls from 0.079 m to 0.008 m, and the same drive is found in 80 iterations.

Section 4.3 ended on the long lever: in single shooting, the first control moves the end of an 8 s drive ten times as far as the same change within a 1.6 s piece.

The idea. Instead of one long shot, fire several short ones. Cut the 8 s into 5 pieces of 1.6 s. Guess where the robot is at the start of each piece, its full state, and simulate each piece from there with its own 6 intervals of controls. At first the pieces do not meet; the solver moves both the controls and the start states until each piece ends exactly where the next one starts (Kelly 2017; Kelly, cannon). Figure 6 runs it; watch the red gaps between the coloured pieces close.

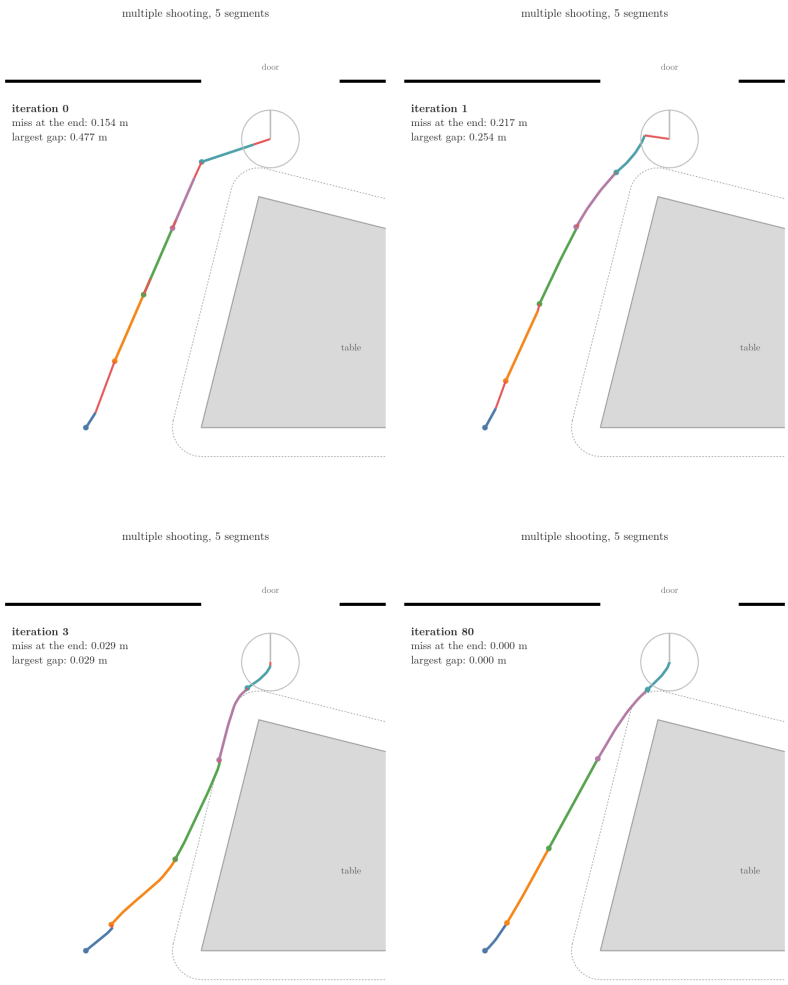


Figure 6: Multiple shooting of the 8 s least-effort drive, one solver iteration per frame (thinning out after the first 12). The drive is cut into 5 pieces of 1.6 s (colours); each is simulated from its own start state (dot), an unknown, first placed on the stop-and-turn route. Red lines are the defects: the gap between a piece's end and the next piece's start, and the miss at the door pose. The largest first gap is 0.477 m; after 80 iterations every gap is 0 and the drive is the one single shooting found

The standard term. Shooting with the trajectory cut into segments, each simulated from its own start state, which is an unknown, with a defect at every join constrained to 0, is **multiple shooting** (G-2892), also called parallel shooting (Kelly 2017).

What changes. The unknowns are now the 60 controls plus 4 start states of 4 numbers, 76 in all, with 4 joins of 4 defects each plus the 4 numbers of the end that must close. In exchange:

- each control acts only within its piece: 0.1 rad/s more turn rate in the first interval moves the end of its 1.6 s piece by 0.008 m, against 0.079 m for the end of the full drive (notebook);

- the pieces are small problems that can be solved in parallel (Kelly, cannon);
- multiple shooting is much more robust than single shooting on hard problems: it converges where single shooting fails (Kelly 2017; Kelly, cannon).

On our short drive both converge: multiple shooting takes 80 iterations to single shooting's 73, from a first guess with gaps of up to 0.477 m, and ends on the same drive with the same effort, 0.217 (notebook). Shorter pieces mean shorter levers; at the limit, every interval is its own piece, and the states at every point are unknowns. Then nothing needs to be simulated at all.

6. Direct collocation: the motion rules as equations at points

> **Key point:** Make the states and the controls at a row of points in time all unknowns, and require between each pair of neighbours that the change in state equals the trapezoid-rule area under its rate of change, so nothing is simulated and limits are simple. Checked on the straight drive, it approaches the known 6.2 s as the points get closer; on the room drive it finds the least time to the door pose, 6.18 s.

Section 5 shortened the pieces of multiple shooting; at one interval per piece every state is an unknown, and the motion rules can be written as equations instead of being simulated (Tedrake, Underactuated ch. 10).

6.1 States at points instead of a simulation

Mark 31 points in time across the drive, 0.267 s apart for the 8 s drive. At each point, the state (4 numbers) and the controls (2 numbers) are unknowns: 186 numbers. Between neighbouring points, the motion rules must hold approximately: the change in each state between the two points must equal how fast it was changing, averaged over the two points, times the time between them. The solver moves all the points at once until every one of those equations holds (Kelly 2017).

The standard terms. Transcription in which the states and the controls at a set of points are all unknowns and the motion rules are enforced as constraints between them is **direct collocation** (G-2893) (Kelly 2017; Tedrake, Underactuated ch. 10). The points are the **knot points** (G-2895), also called collocation points, and each equation that ties neighbouring knot points together through the motion rules is a **collocation constraint** (G-2894) (Kelly 2017).

6.2 The trapezoid rule by hand

The equation between neighbours comes from the area under a straight line. On the straight drive, the first interval of the least-time collocation with 10 intervals lasts 0.6364 s, and in it the speed rises in a straight line from 0 to 0.3182 m/s at 0.5 m/s^2 (notebook). The distance covered is the area under the speed, a trapezoid (here with one side 0, a triangle) (Figure 7):

$$\text{distance} = 0.6364/2 \times (0 + 0.3182)$$

$$= 0.1013 \text{ m}$$

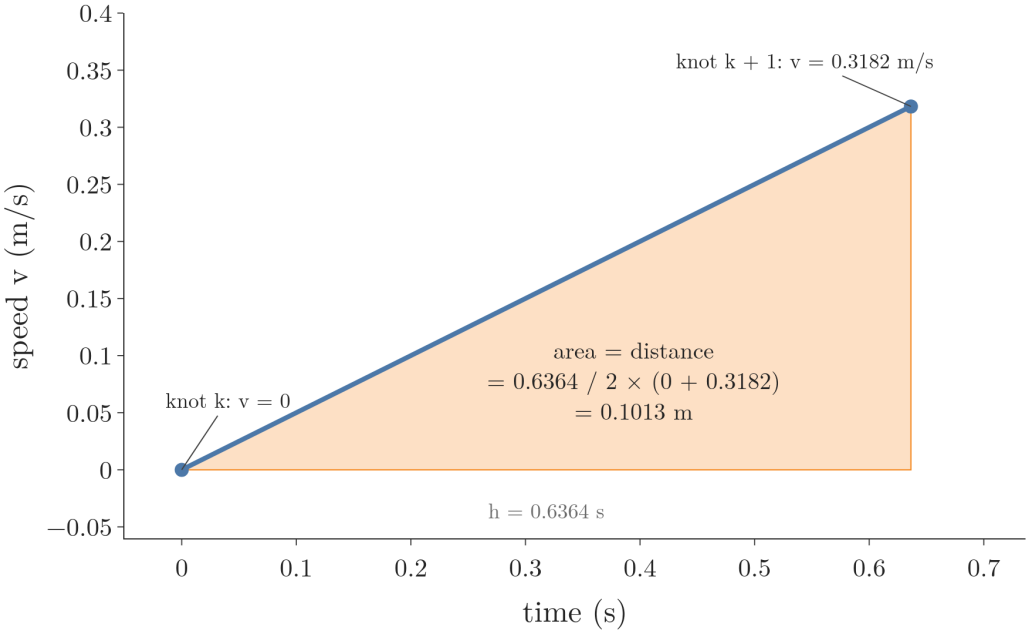


Figure 7: The trapezoid rule on the first interval of the straight drive, 0.6364 s long (10 intervals of the least-time collocation). The speed rises in a straight line from 0 at knot k to 0.3182 m/s at knot k + 1; the distance covered is the area under it (orange), 0.6364 / 2 × (0 + 0.3182) = 0.1013 m, exactly what speeding up at 0.5 m/s^2 for 0.6364 s gives

Constant acceleration gives the same distance, so here the rule is exact, because the speed changes in a straight line:

$$0.5 \times 0.5 \times 0.6364^2 = 0.1013 \text{ m}$$

The same rule written for every state of the robot, with:

- $\mathbf{x}_k$, the state at knot point k (four numbers);
- $\mathbf{u}_k$, the controls at knot point k (two numbers);
- h, the time between knot points, in s (0.267 s for the 8 s drive);
- f_k, the rates of change at knot point k, the four motion rules of Section 2.1 evaluated at $\mathbf{x}_k$ and $\mathbf{u}_k$.

$$\mathbf{x}_{k+1} - \mathbf{x}_k = \frac{h}{2}\left(f_k + f_{k+1}\right)$$

This equation is the collocation constraint of trapezoidal collocation (Kelly 2017, eq. 3.2). The rule assumes the rates change in a straight line between knot points, which makes each

state a parabola between them; where the true rates bend, it is only approximate (Kelly 2017).

6.3 Checking the method on the straight drive

For the straight 3.0 m drive the least time is known: speed up at 0.5 m/s^2 for 1.2 s to 0.6 m/s, cruise, brake, 6.2 s (the fastest straight drive). Trapezoidal collocation with the time as an unknown gives (notebook):

Intervals	Least time found (s)
10	6.364
20	6.239
40	6.209

Figure 8 shows why it is not exact: the speed can only bend at knot points, so the sharp corners where the robot stops speeding up cannot be drawn with 10 intervals. With more knot points the corners sharpen and the time approaches 6.2 s. Adding knot points is the usual way to make a collocation more accurate (Kelly 2017).

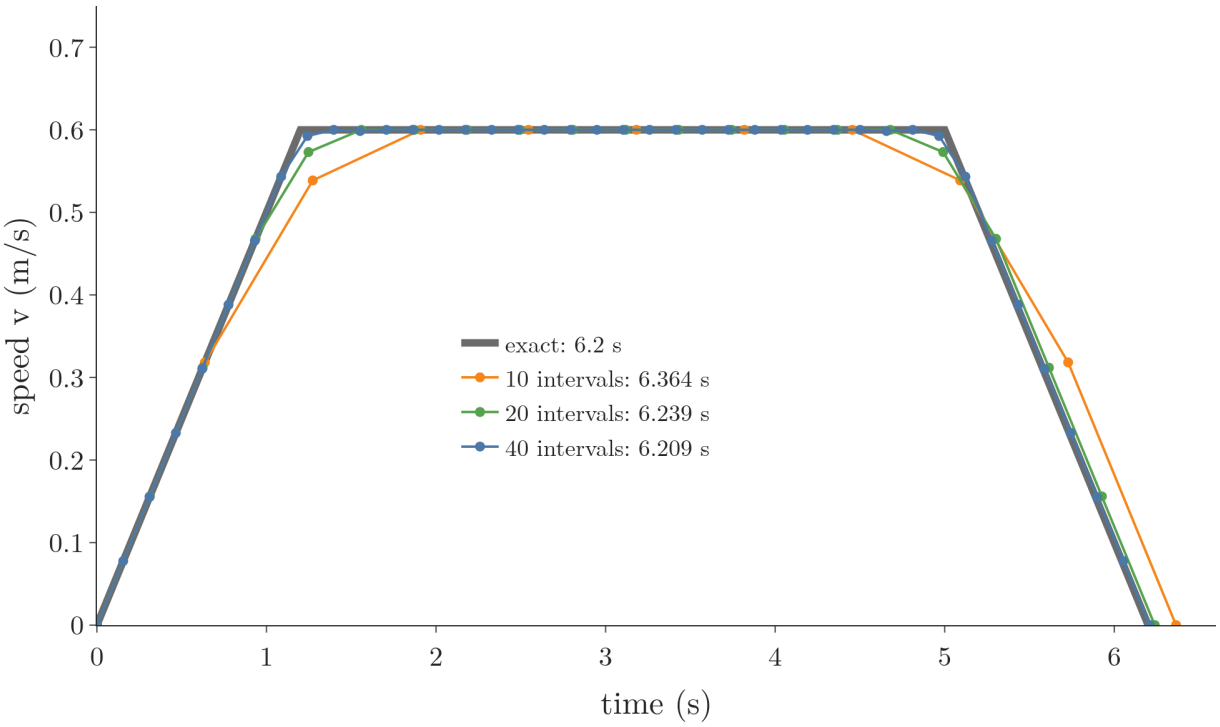


Figure 8: Collocation checked on the straight drive, whose least time is known: 6.2 s (grey: up at 0.5 m/s^2 to 0.6 m/s, cruise, brake). The speed at the knot points of the least-time collocation with 10 (orange), 20 (green) and 40 (blue) intervals. The corners are cut where the knot points are far apart; the times found, 6.364, 6.239 and 6.209 s, approach 6.2 s as the intervals shrink

6.4 The drive to the door

Least effort in 8 s. The first guess puts the 31 knot points on the stop-and-turn route at a steady speed, with the heading turning evenly from 30° to 90°. The guess breaks the motion rules: its largest defect, between where the trapezoid rule says a knot should be and where it is, is 0.121 m (notebook). A planner's route is the usual first guess: trajectory optimisation improves a guess rather than searching from scratch (LaValle §14.7). Figure 9 runs the same least-squares solver as in Section 4.2; watch the knot points slide off the route's corner onto a smooth curve past the table.

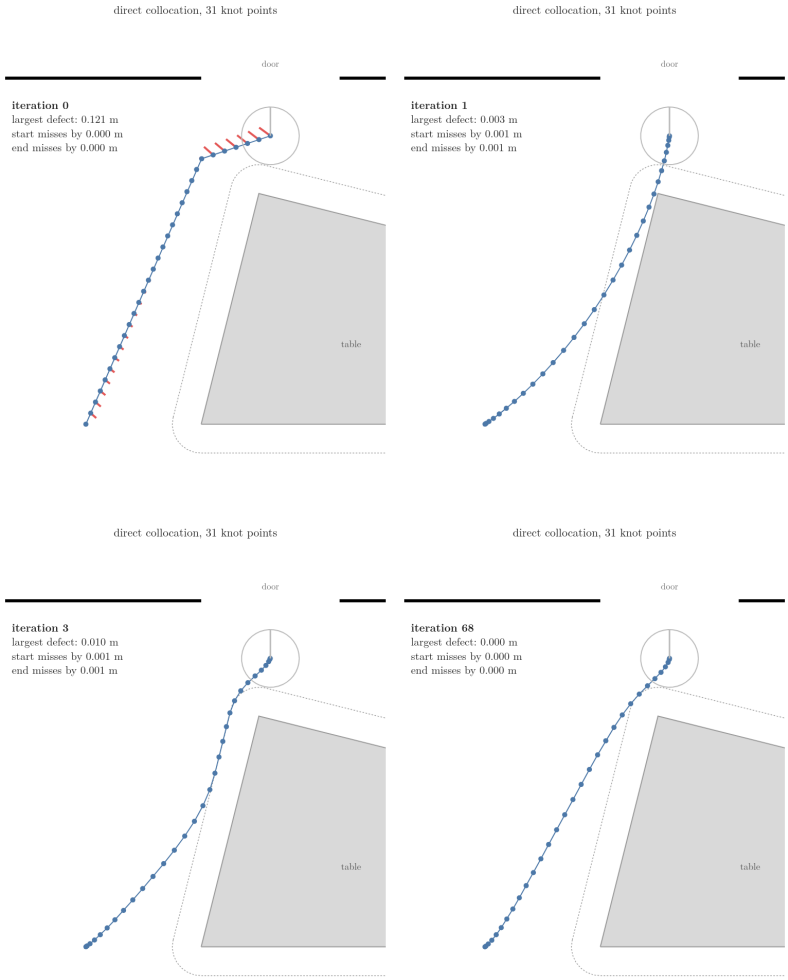


Figure 9: Direct collocation of the 8 s least-effort drive, one solver iteration per frame (thinning out after the first 12). The unknowns are the state and the controls at 31 knot points (dots), 0.267 s apart; nothing is simulated. The first guess puts them on the stop-and-turn route, with the heading turning evenly from 30° to 90°. Red lines show each knot point's defect, the gap between where the trapezoid rule from the previous knot puts it and where it is: the largest is 0.121 m at first and 0 after 68 iterations

After 68 iterations every defect is 0 and the drive matches the shooting one: effort 0.218 against 0.217 (notebook). The small difference comes from the two transcriptions: collocation lets the controls change in a straight line between knot points, shooting holds them constant over each interval. Limits are simple here because the states are unknowns: the wheels, the speed and the table are checked at every knot point and halfway between neighbours, with no simulation in between (Kelly 2017). Collocation suits problems with complicated controls and path constraints (Kelly 2017).

Least time. Now we make the duration T an unknown and the cost, T itself, with a tiny cost on the controls added so that the turn rate does not wander where the time does not care. For this problem we use a solver that keeps every constraint exactly, SciPy's SLSQP, a sequential quadratic programming solver: gradient-based methods of this kind find a locally best trajectory when the first guess is close to one (Lynch and Park §10.7). Figure 10 shows the result over time.

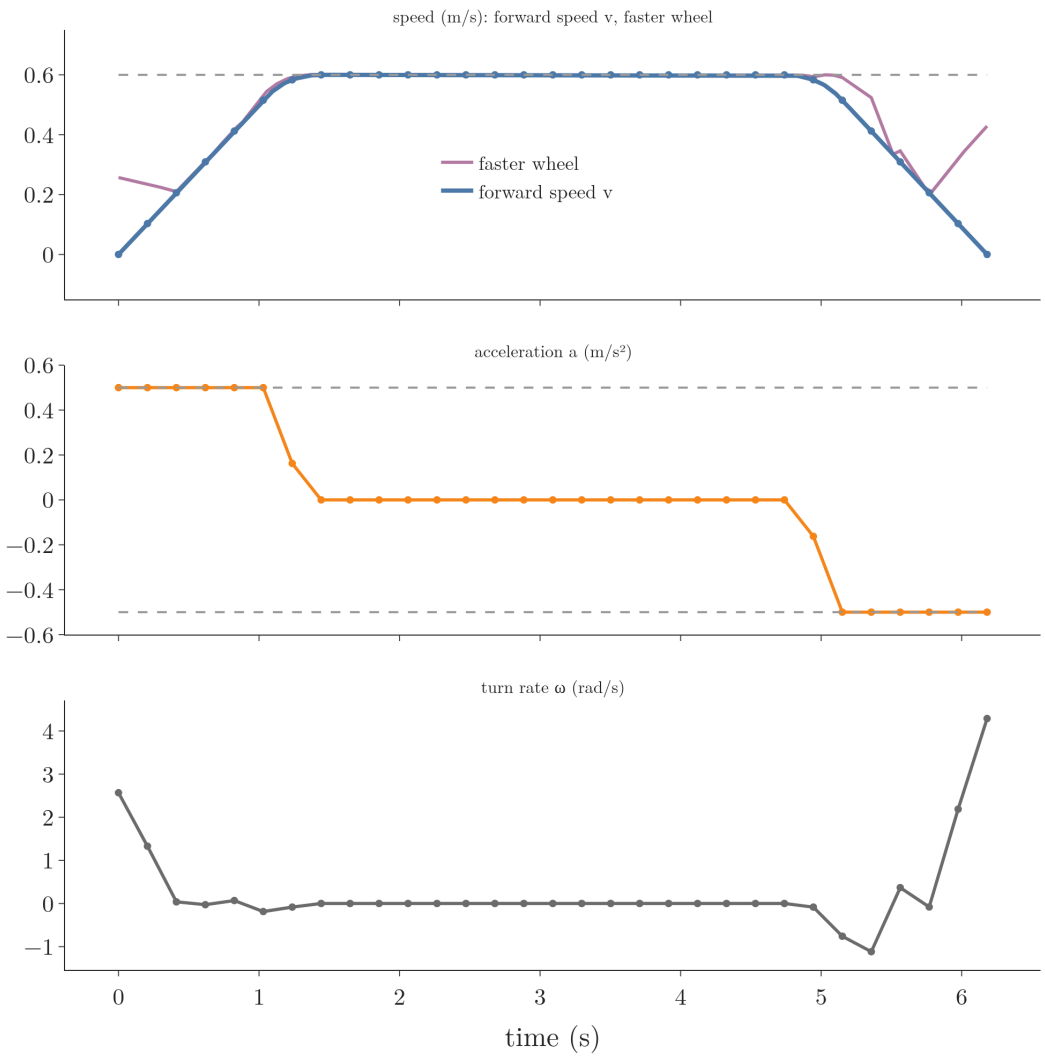


Figure 10: The least-time drive to the door pose found by collocation, 6.18 s, over time; lines: the controls replayed in small simulation steps; dots: the 31 knot points. Top: the forward speed v (blue) and the faster wheel (purple), both at or under the 0.6 m/s limit (dashed). Middle: the acceleration a: $+0.5$ m/s^2 to speed up, 0 while cruising, -0.5 m/s^2 to brake. Bottom: the turn rate ω: a fast turn at the start and at the end, where the speed is low and the wheels allow up to 6 rad/s

The robot reaches the door pose in 6.18 s, the drive of Figure 1 (notebook). Every limit that can be used is used:

- the acceleration sits at $+0.5$ m/s^2 until the speed reaches 0.6 m/s, and at -0.5 m/s^2 to brake;
- the faster wheel sits at its 0.6 m/s limit while the robot cruises;
- the disk passes the table's corner with no room to spare;
- the turns happen at the two ends, where the speed is low and the wheels allow a fast turn.

The opening's straight line, which ignored the table and every turn, could not be driven in less than 6.147 s; the least-time drive needs only 0.034 s more, while the stop-and-turn route needs 1.90 s more (notebook).

Checking between the knot points. The constraints hold at the knot points and halfway between them, and nowhere else is checked. So we replay the found controls in small simulation steps from the start: the replayed drive ends 0.003 m from the door pose and its speed never passes 0.6 m/s (notebook). Collocation enforces the motion rules only at its points, so its error must be checked between them (Kelly 2017, §5.3); the replay is a simple check of that kind.

All the methods so far improve the controls step by step using slopes. The next method takes far better steps by using what the slopes and curvature of the robot's model say about the whole drive.

7. iLQR and DDP: better steps from the model

> **Key point:** iLQR approximates the motion rules near the current drive by straight-line rules and the cost by a bowl, solves that easier problem exactly, from the last step back to the first, then simulates the improved drive; repeated, it needs far fewer iterations than gradient descent. On the straight drive one iteration is exact; on the room drive 13 iterations bring the cost from 1,422,936 to 0.233, where plain gradient descent is still at 174,704.

Shooting in Section 4 needed a solver that aims its steps well; with plain gradient descent the steps are poor. On the room drive's cost, gradient descent from the first shot is still at 2,717.5 after 1000 iterations, against 0.217 for the drive the other methods found (notebook). The robot's model knows more than a slope: how each step of the drive leads to the next.

The idea. Near the current drive, pretend the robot's motion rules are straight-line (linear) rules, which they nearly are for small changes, and that the cost is a bowl, a quadratic like the squared miss of Section 4.1, which it nearly is near its bottom. For that pretend problem the best change of the controls can be worked out exactly, step by step from the end of the drive back to the start: at the last step, how should the control change, and how should it react if the robot is a little off? Then at the step before, knowing the answer for the last step, and so on back to the first. After this sweep from the end to the start, simulate the drive forward with the new controls, correcting by the reactions where the robot drifts from the old drive. The new drive is the start of the next round (Tedrake, Underactuated ch. 10).

The standard terms. Repeating a sweep back from the end that solves the linear-rules, bowl-shaped-cost version of the problem around the current trajectory, and a simulation forward with the result, is **iterative LQR** (G-2896), iLQR; LQR, the linear-quadratic regulator, is the name of the exact method for linear rules and bowl-shaped costs that each sweep back applies (Tedrake, Underactuated ch. 10). **DDP** (G-2897), differential dynamic programming, does the same with curved (second-order) approximations of the motion rules as well; it is almost the same algorithm, and in most cases iLQR converges as quickly (Tedrake, Underactuated ch. 10). Besides the controls, each sweep back also gives

the reactions, which say how to correct the controls when the robot drifts off the trajectory (Tedrake, Underactuated ch. 10).

A check at easy values. On the straight drive the motion rules are already linear (the distance along the line grows at the speed, the speed at the acceleration) and the effort is a bowl, so the pretend problem is the real one, and one iteration should land on the answer. We drive 3.0 m in 10 s with the least effort $\int a^2\,dt$, the miss at the end costing 1,000,000 per m^2. The first guess, $a = 0$, misses by 3.0 m and costs 9,000,000; after one iteration the cost is 0.10812, and the second iteration changes nothing (notebook).

The exact answer is known for this problem: the least-effort move from rest to rest makes the acceleration fall in a straight line (Kelly 2017, §2.2, for a move of unit distance in unit time), and the line scales to any distance and time. With:

- d, the distance, 3.0 m;
- T, the time, 10 s;
- $a(t)$, the acceleration at time t, in m/s^2.

$$a(t) = \frac{6d}{T^2}\left(1 - \frac{2t}{T}\right)$$

With our numbers, and the effort it costs:

$$a(t) = 0.18\,(1 - 0.2\,t)$$

$$\int_0^{10} a^2\,dt = 0.0324 \times \frac{10}{3}$$

$$= 0.108$$

The middle line uses the area under the squared bracket over the 10 s, a parabola:

$$\int_0^{10} (1 - 0.2\,t)^2\,dt = \frac{10}{3}$$

iLQR's 0.10812 differs from 0.108 only because it holds the acceleration constant over 30 intervals; its first interval holds 0.1742 m/s^2, the exact value at that interval's middle, 0.174 m/s^2 (notebook).

The room drive. The robot's real rules turn, so the pretend problem is only close, and iLQR needs several rounds. The run starts from the same first shot as single shooting, with the same cost: the effort plus penalties of weight 1,000,000 for missing the door pose, breaking a wheel limit or touching the table (iLQR handles constraints as penalties, Tedrake, Underactuated ch. 10). Figure 11 runs it next to plain gradient descent on the same cost.

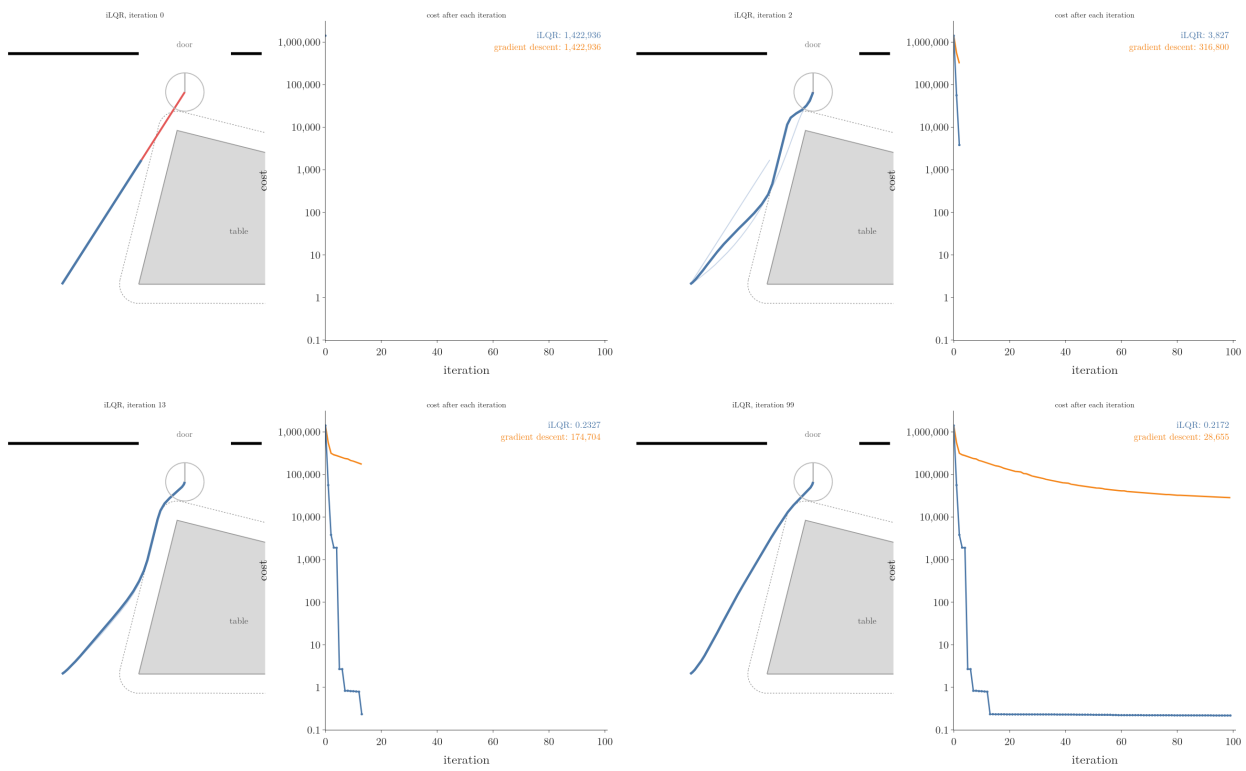


Figure 11: iLQR on the 8 s least-effort drive, one iteration per frame (thinning out after 15), from the same first shot as single shooting. Left: the drive after each iteration (blue; earlier ones pale) and the miss at the end (red). Right: the cost after each iteration, on a scale where each grid line is 10 times the one below it: iLQR (blue) against plain gradient descent on the same cost from the same start (orange). After 13 iterations iLQR is at 0.2327 and gradient descent at 174,704; after 99 iterations iLQR is at 0.2172, the drive the other methods found

After 13 iterations the cost falls from 1,422,936 to 0.2327, within 7 percent of iLQR's final 0.2172, reached at iteration 99 on the same drive as shooting and collocation (notebook). Gradient descent on the same cost stands at 174,704 after 13 iterations and 2,717.5 after 1000. iLQR's steps use the curvature of the cost along the whole drive, much as Newton's method (G-1321) uses the curvature of a function: the update is second order (Tedrake, Underactuated ch. 10).

Shooting, collocation and iLQR all cut the problem into numbers first and then optimise. There is another way round.

8. Direct and indirect methods

> **Key point:** Every method so far discretises first and then optimises: a direct method. An indirect method first derives the conditions any optimal trajectory must satisfy, Pontryagin's minimum principle, and then solves them; for the straight drive with no

speed limit they say full acceleration, then full braking, with one switch.

Sections 4 to 7 all turned the trajectory into a list of numbers before optimising it.

The standard terms. A method that discretises the trajectory optimisation problem into a list of numbers and then optimises them is a **direct method** (G-2898) (Kelly 2017). A method that first works out, by calculus, the conditions an optimal trajectory must meet, and then discretises and solves those conditions, is an **indirect method** (G-2899) (Kelly 2017). In short: a direct method discretises, then optimises; an indirect method optimises, then discretises (Figure 12).

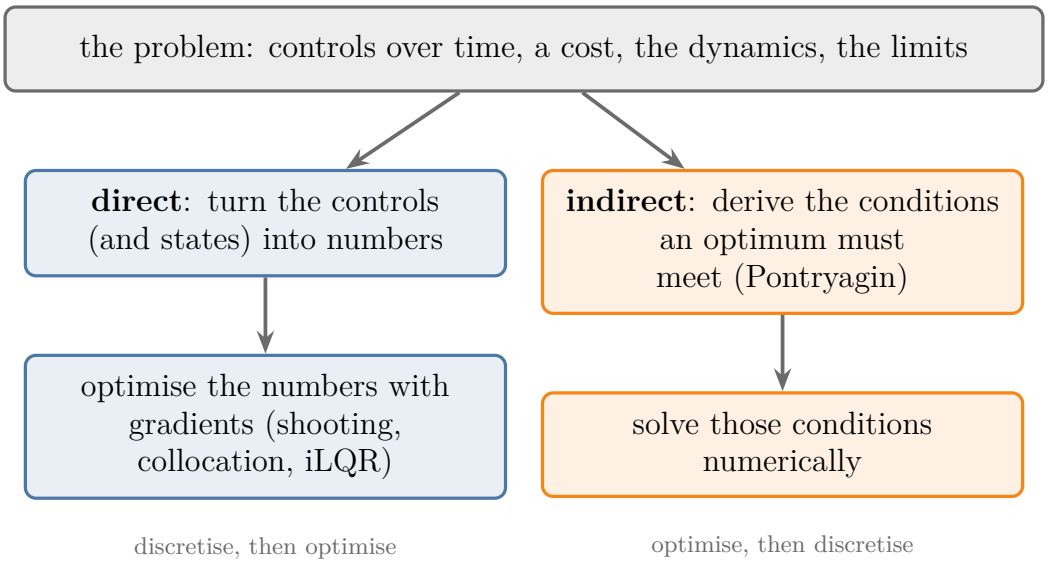


Figure 12: Two ways to solve a trajectory optimisation problem. Direct (blue): turn the controls, and maybe the states, into numbers, then optimise the numbers with gradients, as shooting, collocation and iLQR do: discretise, then optimise. Indirect (orange): derive the conditions an optimum must meet (Pontryagin's minimum principle), then solve those conditions numerically: optimise, then discretise

The conditions come from **Pontryagin's minimum principle** (G-2900): along an optimal trajectory, the controls at each moment minimise a certain function of the state, the controls and a second set of variables (the adjoint variables) that run backwards in time. The principle gives necessary conditions: every optimal trajectory meets them, but a trajectory that meets them need not be optimal (LaValle §15.2.3; Tedrake, Underactuated ch. 10).

On the straight drive. For a least-time drive whose only control is an acceleration between -0.5 and 0.5 m/s^2, with no speed limit, the principle says the acceleration is always at one of its two limits and switches at most once (LaValle §15.2.3, Example 15.4). To cover 3.0 m from rest to rest, the robot accelerates at 0.5 m/s^2 for the first 1.5 m and brakes for the last 1.5 m. With:

- $t_{1/2}$, the time for each half, in s.

$$1.5 = 0.5 \times 0.5 \times t_{1/2}^2$$

$$t_{1/2} = \sqrt{6}$$

$$= 2.449 \text{ s}$$

$$\text{top speed} = 0.5 \times 2.449$$

$$= 1.22 \text{ m/s}$$

So the drive would take 4.90 s, but at 1.22 m/s the wheels would break their 0.6 m/s limit. With the speed limit added, the drive gains a cruise at 0.6 m/s and takes the 6.2 s of Section 6.3.

The trade. An indirect method is generally more accurate and gives a more reliable error estimate, because its conditions are derived exactly. But the conditions are hard to derive for most robots, it needs a good first guess, including guesses for the adjoint variables, which have no physical meaning to guess from, and it converges from a smaller range of first guesses (Kelly 2017). Indirect methods came first and are still used more in aerospace; direct methods are newer, easier to set up, and more widely used in robotics (Cornell talk).

9. Summary

Method	Unknowns (8 s drive)	How the motion rules are kept	Iterations	Why use it
single shooting	60 controls	simulated from the start	73	few unknowns; simple
multiple shooting	60 controls + 4 start states	simulated per piece, joins as defects	80	short levers, more robust
direct collocation	state and controls at 31 knot points	trapezoid rule as equations	68	no simulation; limits easy
iLQR	60 controls	simulated; linearised backwards	99 (13 to within 7 percent)	few, well-aimed steps; corrections for drift

All four find the same 8 s drive, effort 0.217 (0.218 for collocation, whose controls change in straight lines between knot points).

- The robot reaches the door pose in 6.18 s instead of the 8.08 s of stopping and turning along a route, because trajectory optimisation chooses the controls over time directly: it turns while speeding up, cruises at the wheel limit and passes the table's corner with no room to spare, within 0.034 s of the 6.147 s a straight line would need if the table were not there.
- A **trajectory** (G-2886) is the state at every moment and the controls that produce it, so it says when and how, not only where.
- **Trajectory optimisation** (G-2887) finds the controls (and the duration) that minimise a cost such as time or effort while the motion rules, the start and the end, and every limit hold, because one good trajectory is affordable where a rule for every state is not.
- **Transcription** (G-2888) turns the functions into a list of numbers, here 60, so that an optimiser can search over them.
- **Single shooting** (G-2890) simulates from the start and closes the **defect** (G-2891) at the end with gradients; it is simple, but an early control has a long lever (0.079 m per 0.1 rad/s against about 0.0001 m for the last), which makes long drives hard.
- **Multiple shooting** (G-2892) simulates short pieces from unknown start states and closes the gaps, so each lever is short (0.008 m) and the method is more robust.
- **Direct collocation** (G-2893) makes the states at **knot points** (G-2895) unknowns and the **collocation constraints** (G-2894) equations, so nothing is simulated and limits are easy; its accuracy grows with the knot points, 6.364, 6.239, 6.209 s towards the true 6.2 s.
- **iLQR** (G-2896) and **DDP** (G-2897) solve a linear, bowl-shaped version of the problem exactly in a sweep from the end of the drive back to the start, so their steps are second order: one iteration on a linear problem, 13 to within 7 percent on the room drive,

where gradient descent's cost is still 750,000 times higher.
- **Direct methods** (G-2898) discretise and then optimise; **indirect methods** (G-2899) solve the necessary conditions of **Pontryagin's minimum principle** (G-2900), which are more accurate but hard to derive and to start, so robotics mostly uses direct methods.

The robot can now compute a fast, smooth drive that keeps its limits. Still missing: the drive is computed once and played open loop, as a list of commands over time. A real robot slips and drifts, so it ends away from the door pose; following a trajectory needs feedback, which measures where the robot is and corrects the commands, the subject of the next chapter, feedback control.

10. Sources

Built from

- Matthew Kelly, "Introduction to Trajectory Optimization", a talk with slides given at Cornell University, YouTube, https://www.youtube.com/watch?v=wlkRYMVUZTs (Cornell talk)
- Kelly, M. (2017). "An introduction to trajectory optimization: how to do your own direct collocation". *SIAM Review* 59(4), 849–904, https://doi.org/10.1137/16M1062569, free PDF from the author (Kelly 2017)
- Kelly, M. "Cannon example: single shooting, multiple shooting, direct collocation", tutorial page, https://www.matthewpeterkelly.com/tutorials/trajectoryOptimization/canon.html (Kelly, cannon)
- Lynch, K. M. and Park, F. C. (2017). *Modern Robotics: Mechanics, Planning, and Control*. Cambridge University Press. §10.7 "Nonlinear optimization". Free preprint: http://modernrobotics.org (Lynch and Park §10.7)
- Tedrake, R. *Underactuated Robotics*, ch. 10 "Trajectory optimization" (direct transcription, direct shooting, direct collocation, Pontryagin's minimum principle, iterative LQR and DDP), https://underactuated.mit.edu/trajopt.html (Tedrake, Underactuated ch. 10)
- LaValle, S. M. (2006). *Planning Algorithms*. Cambridge University Press. §14.7 "Gradient-based trajectory optimization" and §15.2.3 "Pontryagin's minimum principle", Example 15.4. Free online: https://lavalle.pl/planning/ (LaValle §14.7, §15.2.3)

Other references

- Northwestern Robotics, "Modern Robotics, Chapter 10.7: Nonlinear Optimization", YouTube, https://www.youtube.com/watch?v=-z6Rlei8iPE: the video version of Lynch and Park §10.7
- Virtanen, P. et al. (2020). "SciPy 1.0: fundamental algorithms for scientific computing in Python". *Nature Methods* 17, 261–272: the solvers `least_squares` (Gauss–Newton with a trust region) and `minimize(method="SLSQP")` used in the notebook (Virtanen et al. 2020)

11. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Trajectory (G-2886)	The state of a robot at every moment of a time interval, together with the controls that produce it, such as the 6.18 s drive to the door pose; unlike a path, it says when the robot is where and how it is commanded.
Trajectory optimisation (G-2887)	Finding the controls over time (and the duration, if free) that minimise a cost such as time or effort while the motion rules, the start and end states and every limit hold; it computes one good trajectory where a rule for every state would be too large.
Transcription (G-2888)	Turning a trajectory optimisation problem, whose unknowns are functions of time, into one whose unknowns are a finite list of numbers, such as 60 control values for an 8 s drive cut into 30 intervals, so that an optimiser can search over them.
Single shooting (direct shooting) (G-2890)	Trajectory optimisation that keeps only the controls as unknowns, finds the trajectory by one simulation from the start, and adjusts the controls until the end meets the goal; simple, but an early control moves the end far more than a late one.
Defect (G-2891)	The gap between where a simulation or an integration rule says the trajectory should be and where the unknowns put it, such as a shot ending 1.048 m short of the door pose; the solver drives every defect to 0.
Multiple shooting (parallel shooting) (G-2892)	Shooting with the trajectory cut into segments, each simulated from its own start state, which is also an unknown, with a defect at every join constrained to 0; each control then acts on one short piece, which makes it more robust than single shooting.

Term	Meaning
Direct collocation (G-2893)	Transcription in which the states and controls at a row of points in time are all unknowns and the motion rules are enforced as equations between neighbouring points; nothing is simulated, so limits along the way are easy to add.
Knot point (collocation point) (G-2895)	One of the points in time at which collocation keeps the state and controls as unknowns, such as 31 points 0.267 s apart over an 8 s drive; more knot points make the solution more accurate.
Collocation constraint (trapezoidal) (G-2894)	The equation that ties neighbouring knot points through the motion rules; in trapezoidal collocation the change in state equals the time step times the average of the rates at the two points, $x_{k+1} - x_k = \frac{h}{2}(f_k + f_{k+1})$.
iLQR (iterative LQR) (G-2896)	Trajectory optimisation that approximates the motion rules near the current trajectory by linear rules and the cost by a bowl, solves that version exactly in a sweep from the last step back to the first, and simulates the result forward; its steps are second order, so it needs far fewer iterations than gradient descent, and it also gives feedback gains.
DDP (differential dynamic programming) (G-2897)	iLQR with curved (second-order) approximations of the motion rules as well as of the cost; almost the same algorithm, and in most cases no faster to converge than iLQR.
Direct method (trajectory optimisation) (G-2898)	A method that first discretises the trajectory problem into a list of numbers and then optimises them, such as shooting, collocation and iLQR; easier to set up, and the usual choice in robotics.
Indirect method (trajectory optimisation) (G-2899)	A method that first derives by calculus the conditions an optimal trajectory must meet, then discretises and solves those conditions; more accurate, but the conditions are hard to derive and it needs a good first guess.

Term	Meaning
Pontryagin's minimum principle (G-2900)	The necessary conditions for an optimal trajectory: at each moment its controls minimise a function of the state, the controls and adjoint variables that run backwards in time; every optimum meets them, but meeting them does not prove optimality. Indirect methods solve them.
State x_t (G-2314)	The quantities a robot keeps track of at time t because they decide what happens next, such as its pose, its speeds and moving things around it; it is what localization and filtering estimate.
Control input u (G-2298)	The numbers we choose to drive a robot, such as $u = (v, \omega)$; the model turns them into motion.
Kinematic model (G-2297)	Equations that give how a robot's configuration changes from its speed inputs alone, ignoring masses and forces, such as $\dot{x} = v\cos\theta$, $\dot{y} = v\sin\theta$, $\dot{\theta} = \omega$; used to predict motion at low speed.
Integration (G-957)	Finding the area under a curve by adding up infinitely many thin strips.
Constraint (G-456)	A condition the solution must satisfy; in SVM, $y_i(w^T x_i + b) \geq 1$ for every training point.
Policy (G-1512)	The agent's rules for which action to take.
Gradient descent (G-862)	Finding the lowest point of a function by repeated small steps downhill.
Learning rate (η) (G-1068)	How strongly each update changes the model; in gradient descent, the number the slope is multiplied by to get the step size.
Chain rule (G-371)	To differentiate a function of a function, multiply the outer derivative by the inner derivative.
Euler step (G-2299)	One step of predicting motion: hold the rates fixed for a short time Δt and add rate times Δt to each quantity; repeated, it traces a path, more accurately the smaller Δt is.

Term	Meaning
Jacobian (G-980)	The table (matrix) of all first partial derivatives of a function with several inputs and outputs, one row per output and one column per input, $J_{ij} = \partial f_i / \partial x_j$; it shows how every output changes with every input near a point.
Newton's method (G-1321)	A way to find a minimum: at the current point, fit a bowl (the second-order Taylor polynomial, from the first and second derivatives), jump to its lowest point, and repeat; the step is $\boldsymbol{\delta} = -H^{-1} \nabla f^{\mathsf{T}}$. XGBoost uses one such step for every tree.